CPSC 4900 Senior Project Proposal

SeeForMe: Context-Aware Visual Assistance for Blind and Low-Vision Users

Alláh-u-Abhá Rodrigues | CPSC 4900, Fall 2026
Code: github.com/AllahuRodrigues/SeeForMe

1. Introduction and Research Question

During the summer at the Yale Hacker House in San Francisco, I was surrounded by student founders working on practical accessibility problems. A housemate building voice-first tools for blind and low-vision users described how a visually grounded task that seemed simple to a sighted person could become difficult when the assistant could recognize content but could not decide what information mattered at that moment. That experience motivated the central problem of this project: modern perception systems can see many things at once, but spoken feedback is sequential and limited.

SeeForMe is a smartphone-based visual assistance prototype for blind and low-vision (BLV) users. Its research question is: can predictive, context-aware cue scheduling reduce unnecessary spoken information and late warnings while preserving timely communication of action-relevant visual events? Instead of narrating every detected object, SeeForMe treats speech as a scarce communication channel and decides whether to remain silent, speak a cue, interrupt an existing cue, or fall back to haptic feedback.

The project began with CACS (Context-Aware Cue Selection), a weighted ranking rule. On a timing-aware synthetic benchmark, CACS was quiet but late: 61% of its critical warnings missed their safe window because proximity was inferred from bounding-box size. I therefore built P-CACS (Predictive CACS), which uses metric depth, relative motion, time-to-contact, path-crossing probability, announcement memory, and a speech budget. The Python prototype, CampusBench simulator, learned scheduler, browser demo, and evaluation scripts are implemented; the remaining semester work is a frozen transfer evaluation on annotated campus video.

2. Related Work and Research Gap

Commercial tools already provide substantial visual assistance. Seeing AI includes image description and a World mode that announces nearby objects with spatial audio [1]. Be My AI describes captured visual content and supports follow-up questions [2]. Google Lookout offers continuous Explore output and a Find mode that can provide direction and distance guidance to a selected object category [3]. Apple Magnifier can detect nearby people, doors, and furniture and provide speech, sound, or haptic feedback [4]. Mobilio combines smartphone sensing, machine learning, and personalized audio guidance; in a 14-participant study it reduced navigation time by 13% and environmental contacts by 41% relative to Google Maps plus a white cane [5].

User studies show why simply producing richer descriptions is not enough. A two-week diary study with 16 BLV participants found that AI scene descriptions were often used to inspect known objects and, unexpectedly, to avoid hazards, while satisfaction and trust remained modest [6]. In 418 conversational visual-question-answering interactions with 11 blind participants, responses were typically much longer than users' inputs and distance estimation remained difficult [7]. Earlier work likewise found that the content people want described depends strongly on the task and information goal [8].

Recent research is increasingly close to the scheduling problem. WorldScribe adapts live descriptions to user intent and visual context [9]. WalkVLM-LR explicitly reduces output and temporal redundancy with learned risk assessment [10]. ForeSightGuide predicts dynamic hazards and suppresses objects unlikely to threaten the user [11]. Time-Aware Assistive Navigation directly studies what an assistive agent should say and when it should remain silent [12]. Salience-LLaVA orders generated content by importance [13]. One VocalEyes system combines real-time distance-aware detection with contextual VLM descriptions [14], while a separate 2026 system explicitly adds priority-based alerting and "intelligent silence" [15].

SeeForMe therefore does not claim that prioritization or silence is itself new. Its narrower contribution is to isolate cue scheduling as a standalone decision layer between perception and speech. The same structured event stream is given to heuristic, learned, and baseline schedulers, so the scheduling policy can be evaluated separately from the detector or language model using explicit communication windows, preemption, audio occupancy, and frozen transfer to real video.

3. Contributions

1. A formal cue-scheduling problem in which the system chooses among SILENCE, SPEAK, INTERRUPT, and HAPTIC under an explicit communication budget.
2. CampusBench, a controlled metric simulator in which action-relevant events have earliest-useful, ideal, and latest-safe communication times, severity, and minimum message content.
3. A comparison of interpretable and learned scheduling policies under identical perception inputs, followed by a frozen real-video transfer evaluation with predefined hypotheses and failure criteria.

4. Problem Formulation

Consider a blind user walking through a hallway while the camera detects a person, chair, door, EXIT sign, backpack, and poster on each frame. Four problems arise if every high-confidence detection is announced: detector confidence is not the same as importance; persistent objects are repeated; speaking about one object blocks other cues; and a safety warning delivered after the last useful reaction time is effectively a miss.

At each decision step t, the tracker produces a set of candidate events. Each event has a class, confidence, estimated distance, relative speed, time-to-contact (TTC), path-crossing probability, uncertainty, novelty, goal relevance, and candidate message slots. Speaking cue i takes d_i seconds, approximated in CampusBench from message length. For ground-truth event i, the benchmark defines a communication window from earliest-useful to latest-safe time, an ideal time, severity s_i, and the minimum information needed for a useful message.

$$J = \Sigma_{i \in S} s_i \varphi(t_i) - c_{fp} n_{fp} - c_{late} \Sigma_{i \in M} s_i - c_{occ} T_{speech}$$

Here S is the set of events spoken inside their valid window, M is the set of late or missed events, $\varphi(t_i)$ rewards timely delivery, n_{fp} counts off-window, repeated, or unnecessary cues, and T_{speech} is total speaking time. The benchmark uses $c_{fp} = 0.3$, $c_{late} = 0.5$, and $c_{occ} = 0.15$ per second.

P-CACS assigns each candidate an interpretable utility combining hazard, urgency, actionability, text importance, goal relevance, novelty, detector/depth uncertainty, message cost, and the opportunity cost of blocking the audio channel. The urgent TTC threshold is set to 1.5 s for the benchmark, with a tuned horizon of 9.7 s. SPEAK requires sufficient utility and speech-budget credit; INTERRUPT requires a more severe event and either a clear utility advantage or a critical hazard within the urgent horizon; HAPTIC is reserved for confirmed on-path hazards when speech is unavailable. The composer then produces only the slots needed for action, for example, “Stairs down, about 3 meters. Handrail right.”

5. System Design and Data

5.1 Architecture

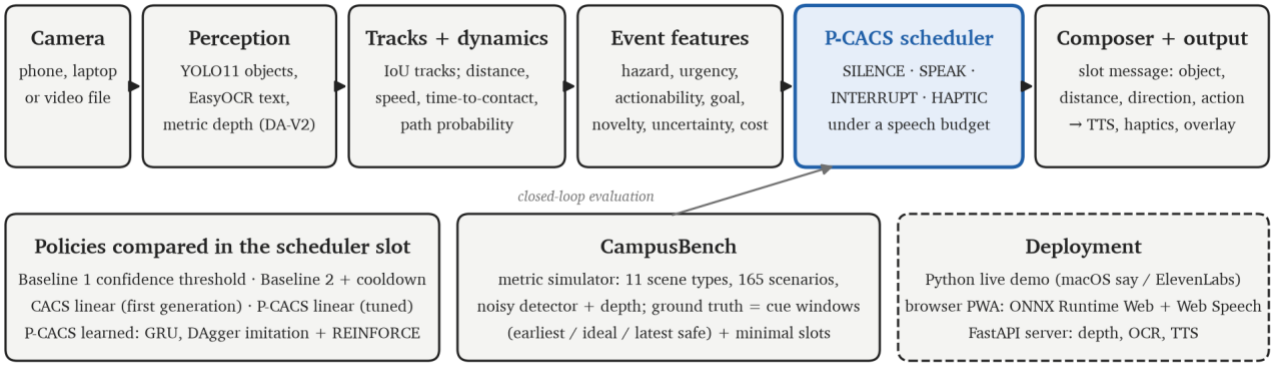


Figure 1. SeeForMe architecture. Perception produces tracked metric events; the scheduler slot can run a baseline, linear P-CACS, or the learned policy; CampusBench (the environment) evaluates each policy in closed loop.

5.2 Implementation

The perception backbones are off the shelf; the scheduling, dynamics, benchmarking, and evaluation layers are implemented for this project. Detection uses Ultralytics YOLO11n/s, OCR uses EasyOCR, and metric depth uses Depth Anything V2 metric-small [16]. A simple IoU tracker maintains identity, announcement memory, distance, relative speed, time-to-contact, and path-crossing probability. Research code is in Python/PyTorch. The browser prototype uses Vite and TypeScript, ONNX Runtime Web for YOLO, Web Speech or optional ElevenLabs output, and a FastAPI server on the user's laptop for depth, OCR, and speech synthesis.

5.3 CampusBench and real-video data

CampusBench is a metric simulator in which objects move relative to a user walking at 1.1 m/s. Bounding boxes come from a pinhole camera model, while noisy detector and noisy depth outputs generate the observations seen by each scheduler. It contains 11 scene types-including hallways, stairs, doors, crossings, crowds, goal finding, poor camera handling, negative scenes, and changing hazards-split into 66 training, 33 validation, and 66 held-out test scenarios. The current benchmark contains 831 events that should be spoken. Events that become observable only after their latest safe time are flagged as sudden and reported separately.

The real-video evaluation will use about 24 chest-mounted phone clips of roughly 90 seconds each (about 36 minutes) across hallways/stairwells, a courtyard, street crossings, and offices/doors, targeting at least 150 annotated communication windows. For each window I will annotate category, earliest useful time, ideal time, latest safe time, severity, target, and minimum message content, then freeze the labels before running any scheduler. Two sighted volunteers will independently annotate 20% of the material without seeing system output. Agreement will be reported as Cohen's κ for categorical event attributes on matched events and temporal IoU for timing. These annotations are a proxy for user preference; a future BLV participant study is required to validate that proxy.

6. Scheduling Policies and Learning

The linear P-CACS weights and thresholds are tuned by random search on the CampusBench training split using the communication score. The learned scheduler is a GRU over each track's previous eight decision steps (29 signals per step) with heads for the action, urgency, and message slots. It is first trained by imitation of an annotation-driven reference scheduler with three rounds of DAgger (43k visited states), then fine-tuned in closed loop with REINFORCE using the communication score as reward. Both learned and linear policies are trained only on CampusBench and then frozen for real-video testing.

Baselines are designed to separate the value of scheduling from the value of extra perception. Baseline 1 speaks detections above a confidence threshold. Baseline 2 adds a 5 s per-label cooldown. Baseline 3, the primary comparator, receives the same depth and motion features as P-CACS and speaks any tracked object

within 3 m or with time-to-contact (TTC) below 3 s, using the same cooldown. First-generation CACS is an additional comparator.

7. Evaluation Plan and Success Criteria

Primary metrics are action-weighted recall, late-critical rate, lead time before the deadline, cue precision, audio occupancy, cues per minute, silence precision, minimum-message ratio, latency, and communication regret against the annotation-driven reference policy. Threshold sweeps will produce load-versus-recall curves, and ablations will remove one P-CACS signal or rule at a time. Real clips are a frozen transfer test: no scheduler is retrained on them. I will report clip-clustered bootstrap intervals, paired per-clip comparisons, and site-stratified results to show whether performance changes across hallways, crossings, offices/doors, and the courtyard.

Two hypotheses are fixed before the real-video run. H1 (load reduction): the relative change in cues per minute, (P-CACS − Baseline 3) / Baseline 3, should have a 95% interval entirely below −0.50. H2 (non-inferiority of timely communication): the lower bound of the P-CACS minus Baseline 3 action-weighted-recall difference should be above −0.10, and the upper bound of the late-critical-rate difference should be below +0.05. If the planned sample cannot support these margins, I will report the achieved interval widths; if either hypothesis fails, that is the result.

8. Preliminary Results

Everything above is implemented except Baseline 3 and the real recordings. The repository currently reports 37 passing tests, and make campus-all reproduces the synthetic benchmark. Table 1 summarizes the held-out CampusBench test split (66 scenarios × 3 noise seeds).

Scheduler	Recall	Late crit.	Precision	Occupancy	Cues/min	Regret
Baseline 1 (confidence)	0.25	0.27	0.06	0.73	31.8	8.47
Baseline 2 (+ cooldown)	0.26	0.27	0.07	0.64	27.0	7.32
CACS, linear	0.23	0.61	0.25	0.23	8.5	3.20
P-CACS, linear	0.48	0.28	0.55	0.21	7.3	1.41
P-CACS, GRU imitation	0.68	0.18	0.73	0.25	9.0	0.55
P-CACS, GRU + REINFORCE	**0.70**	**0.21**	**0.88**	**0.20**	7.4	**0.11**
Reference scheduler	0.73	0.24	1.00	0.21	7.1	0.00

Table 1. Preliminary CampusBench results on the held-out synthetic test split. Higher recall/precision are better; lower late-critical rate, occupancy, cues/min, and regret are better.

The confidence baselines occupy the audio channel 64-73% of the time while communicating only about one quarter of severity-weighted events within their windows. CACS reduces load but is late on 61% of critical events. Linear P-CACS approximately doubles timely recall relative to CACS at similar airtime. The learned policies perform best on the simulator: GRU + REINFORCE reaches 0.70 recall, 0.88 precision, and 0.20 occupancy, close to the annotation-driven reference policy's 0.73 recall. These results are a unit test of the scheduler, not evidence of real-world effectiveness, because CampusBench and the reference policy encode assumptions about what matters. The real-video transfer is therefore the central empirical test.

Measured perception-to-decision processing latency on my Mac is approximately 48 ms (about 21 FPS) with YOLO11n plus metric depth; the scheduling policy itself takes about 0.01 ms. The separate deployment target is camera-to-audio cue latency below 1 s on the iPhone browser demo.

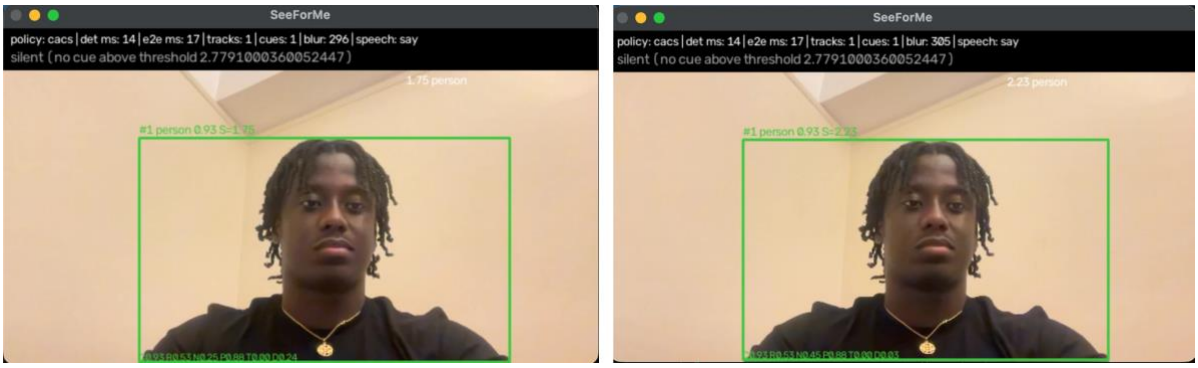


Figure 2. Live demonstration of SeeForMe with Alláh-u-Abhá Rodrigues. The system tracks the subject across consecutive frames and updates distance and cue priority in real time.

10. Timeline and Deliverables

Weeks count from the first week of classes. I budget 7-10 hours per week and meet with my advisor for 30 minutes weekly. P0 = must-have, P1 = important, P2 = stretch.

Weeks	Dates	Milestone / deliverable
1-5	through Oct 4	Done: pipeline, overlay, CampusBench, CACS/P-CACS, depth/dynamics, learned scheduler, browser app, server, revised proposal.
6	Oct 5-11	P0: implement Baseline 3; recruit annotators; record set A; begin annotation.
7	Oct 12-18	P0: record set B; complete primary annotation; freeze labels; first frozen-transfer run.
8	Oct 19-25	P0: progress report (Oct 23) with first real-video results and interval widths.
9	Oct 26-Nov 1	P0: double-annotate 20%; agreement statistics; add clips if intervals are wide. P1: lateral-crossing improvements.
10	Nov 2-8	P0: frozen-transfer evaluation, site-stratified results, ablations, load-recall curves, hypothesis tests; progress presentation.
11	Nov 9-15	P1: iPhone/Mac demo acceptance; decide linear vs learned scheduler. P2: calibration/browser depth.
12	Nov 16-22	Buffer; freeze results; write Methods and Results.
13-14	Nov 23-Dec 6	P1: demo video. P0: poster and full report draft to advisor.
15-16	Dec 7-17	P0: poster session; final report, 250-300 word abstract, code, and course web page.

Core deliverables

- Repository with reproducible experiments, tests, P-CACS schedulers, CampusBench, metrics, and annotation format.
- Annotated real-video dataset (~24 clips, ~36 minutes, 150+ windows) with agreement statistics.
- Frozen-transfer evaluation against Baseline 3, including ablations, site-stratified results, H1/H2, and confidence intervals.
- Browser demo on iPhone/Mac, 2-3 minute demo video, poster, final report, abstract, and course web page.

11. Future Work

The next step is an IRB-approved study with low-vision participants measuring task completion, workload, trust, and preferred verbosity across schedulers. Longer-term extensions include on-device depth, a native Core ML build, spatial audio, glasses-mounted cameras, a small VLM used only to phrase cues while the scheduler controls what and when to communicate, and personalization of scheduling weights from user feedback.

References

[1] Microsoft, “Seeing AI,” mobile application, current feature description. https://apps.apple.com/us/app/seeing-ai/id999062298

[2] Be My Eyes, “How do I use Be My AI?,” Help Center, updated Oct. 14, 2025. https://support.bemyeyes.com/hc/en-us/articles/18133134809105-How-do-I-use-Be-My-AI

[3] Google, “Use Lookout to explore your surroundings,” Android Accessibility Help. https://support.google.com/accessibility/android/answer/9031274

[4] Apple, “Detect doors, people, and furniture around you using Magnifier on iPhone,” Apple Support. https://support.apple.com/en-mide/guide/iphone/iph35c335575/ios

[5] R. Liu and P. Slade, “Improving outdoor navigation for people with blindness using an AI-driven smartphone application and personalized audio guidance,” Nature Biomedical Engineering, 2026, doi:10.1038/s41551-026-01772-x. https://doi.org/10.1038/s41551-026-01772-x

[6] R. E. Gonzalez Penuela, J. Collins, C. Bennett, and S. Azenkot, “Investigating Use Cases of AI-Powered Scene Description Applications for Blind and Low Vision People,” Proc. CHI 2024, 2024, doi:10.1145/3613904.3642211. https://doi.org/10.1145/3613904.3642211

[7] F. Zamiri Zeraati, Y. T. Cao, Y. Qiao, H. Daumé III, and H. Kacorri, “Say It My Way: Exploring Control in Conversational Visual Question Answering with Blind Users,” Proc. CHI 2026, 2026, doi:10.1145/3772318.3791834. https://doi.org/10.1145/3772318.3791834

[8] A. Stangl, N. Verma, K. R. Fleischmann, M. R. Morris, and D. Gurari, “Going Beyond One-Size-Fits-All Image Descriptions to Satisfy the Information Wants of People Who Are Blind or Have Low Vision,” Proc. ASSETS 2021, 2021, doi:10.1145/3441852.3471233. https://doi.org/10.1145/3441852.3471233

[9] R.-C. Chang, Y. Liu, and A. Guo, “WorldScribe: Towards Context-Aware Live Visual Descriptions,” Proc. UIST 2024, 2024, doi:10.1145/3654777.3676375. https://doi.org/10.1145/3654777.3676375

[10] C. Li, Z. Yuan, H. Bi, Z. Jia, and J. Zhang, “Less Redundancy: Boosting Practicality of Vision Language Model in Walking Assistants,” Proc. ICASSP 2026, 2026, doi:10.1109/ICASSP55912.2026.11464758. https://doi.org/10.1109/ICASSP55912.2026.11464758

[11] Z. Wang, X. Li, S. Guo, W. Meng, Q. Liu, and J. Zuo, “ForeSightGuide: An Anticipatory Framework toward Accurate and Low-Redundancy Guidance for the Visually Impaired,” arXiv:2608.18993, 2026. https://arxiv.org/abs/2608.18993

[12] M. Kuribayashi, Z. Shangguan, and E. Ohn-Bar, “Time-Aware Assistive Navigation,” ICRA 2026; arXiv:2609.05596, 2026. https://arxiv.org/abs/2609.05596

[13] J. Liang et al., “Focus Where It Counts: A Salience-Driven Vision-Language Model for Low Vision Assistance,” IEEE Trans. Neural Netw. Learn. Syst., 2026, doi:10.1109/TNNLS.2026.3726274. https://doi.org/10.1109/TNNLS.2026.3726274

[14] K. Chavan, K. Balaji, S. Barigidad, and S. R. Chiluveru, “VocalEyes: Enhancing Environmental Perception for the Visually Impaired through Vision-Language Models and Distance-Aware Object Detection,” Proc. IEEE ICEI 2024, 2024, doi:10.1109/ICEI64305.2024.10912415. https://doi.org/10.1109/ICEI64305.2024.10912415

[15] P. Renukadevi, M. Kumaresan, P. Manikandan, and N. Shivani, “VocalEyes: A Context-Aware Real-Time Assistive Vision System with Priority-Based Audio Guidance,” J. Wireless Mobile Networks, Ubiquitous Computing, and Dependable Applications, vol. 17, no. 2, pp. 663-683, 2026, doi:10.58346/JOWUA.2026.I2.037. https://doi.org/10.58346/JOWUA.2026.I2.037

[16] L. Yang, B. Kang, Z. Huang, Z. Zhao, X. Xu, J. Feng, and H. Zhao, “Depth Anything V2,” Advances in Neural Information Processing Systems 37 (NeurIPS 2024), 2024, doi:10.52202/079017-0688. https://doi.org/10.52202/079017-0688